CAHIER DES CHARGES TECHNIQUE

Projet : Cartes grises

Langages : MySQL – Java

Méthodologie : Agile Kanban (Trello)

Zakaria Saïdouni

BTS SIO – 2024/2025

Sommaire

1. Contexte du projet
    1.1. Présentation du projet
    1.2. Date de rendu du projet
2. Besoins fonctionnels
3. Ressources nécessaires à la réalisation du projet
    3.1. Ressources matérielles
    3.2. Ressources logicielles
4. Gestion du projet
5.Partage Collaboratif sur GitHub
6. Conception du projet
    6.1. Le front-end
        6.1.1. Wireframes
        6.1.2. Maquettes
        6.1.3. Arborescences
    6.2. Le back-end
        6.2.1. Diagramme de cas d'utilisation
        6.2.2. Diagramme d'activités
        6.2.3. Modèles Conceptuel de Données (MCD)
        6.2.4. Modèle Logique de Données (MLD)
        6.2.5. Modèle Physique de Données (MPD)
7. Technologies utilisées
    7.1. Langages de développement Web
    7.2. Base de données
8. Sécurité
    8.1. Protection contre les injections SQL

1. Contexte du projet

1.1. Présentation du projet

Le projet consiste à améliorer l'application utilisée par le service des cartes grises de la préfecture. La Direction des systèmes d'information a demandé une évolution de cet outil afin de mieux répondre aux nouveaux besoins du service.

Notre mission est d'analyser l'application actuelle, de concevoir les améliorations nécessaires, puis de proposer et mettre en place une solution plus performante et adaptée.

1.2. Date de rendu du projet

Le projet doit être rendu au plus tard le 15/01/2025.

2. Besoins fonctionnels

permettre une gestion plus efficace des dossiers de cartes grises (création, modification, suivi).

L'application doit aussi répondre aux nouvelles attentes du service, comme améliorer la rapidité, l'ergonomie et les fonctionnalités existantes.

3. Ressources nécessaires à la réalisation du projet

3.1. Ressources matérielles

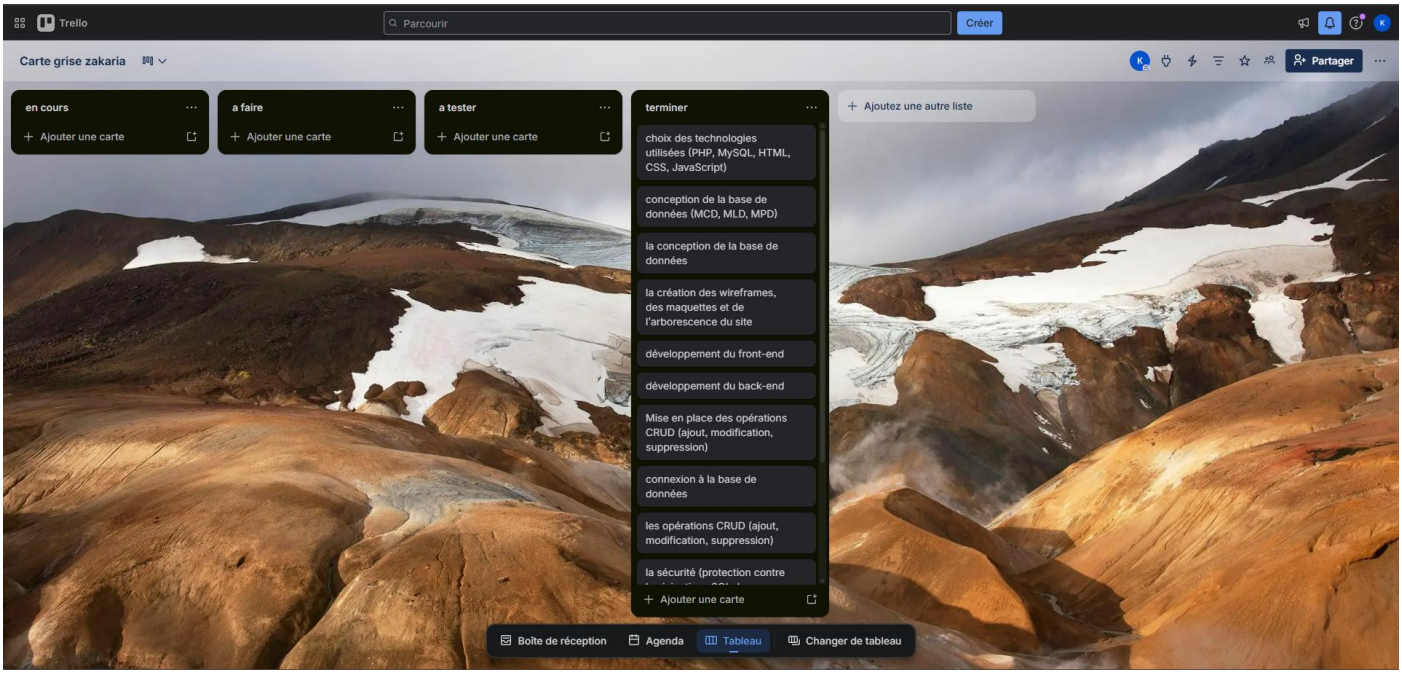
- PC
- Connexion internet
- Serveur local
- Périphérique de travail

3.2. Ressources logicielles

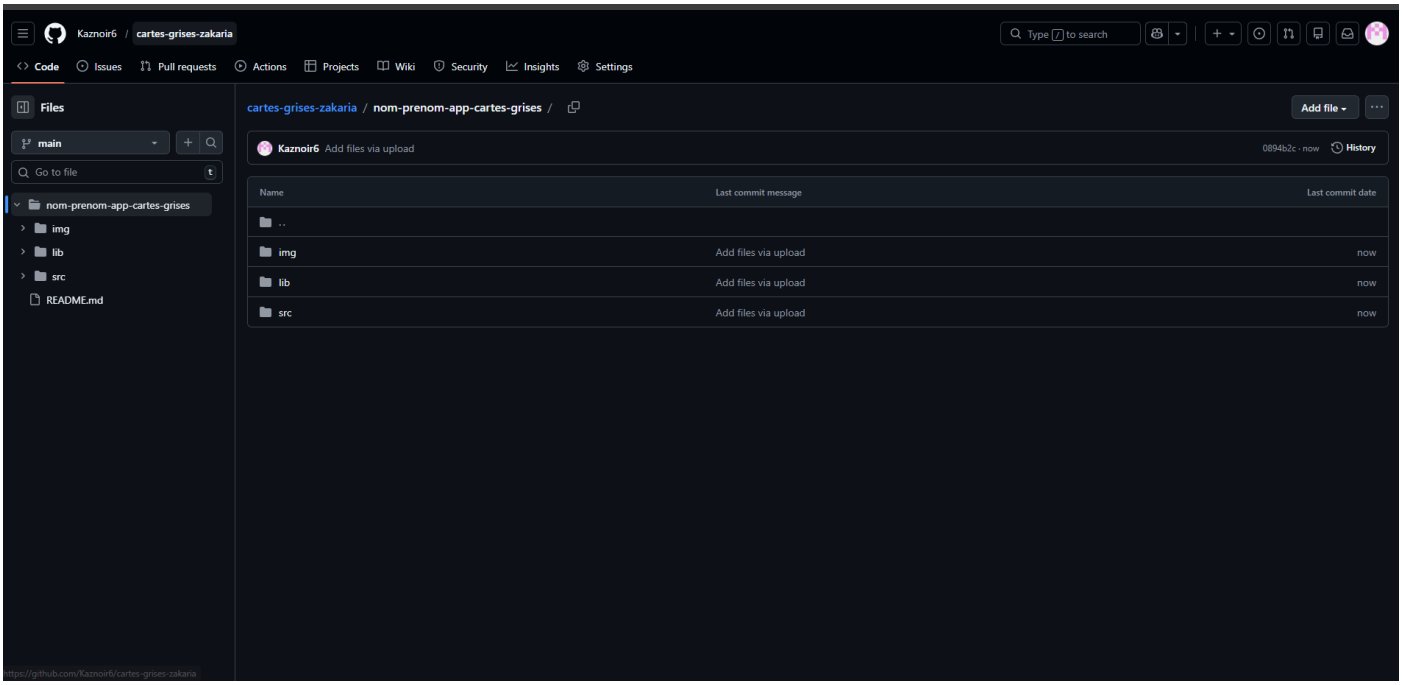
- Visual studio code
- Mysql
- Mamp
- Mocodo
- Java

4. Gestion du projet

Pour réaliser le projet, nous utiliserons la méthode Agile Kanban. Nous utiliserons également l'outil de gestion de projet en ligne Trello.



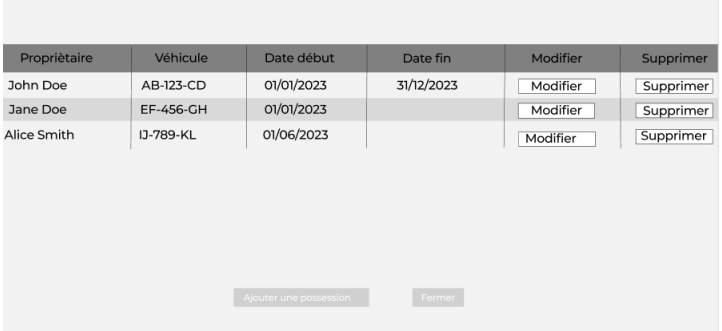
5. Partage collaboratif sur GitHub

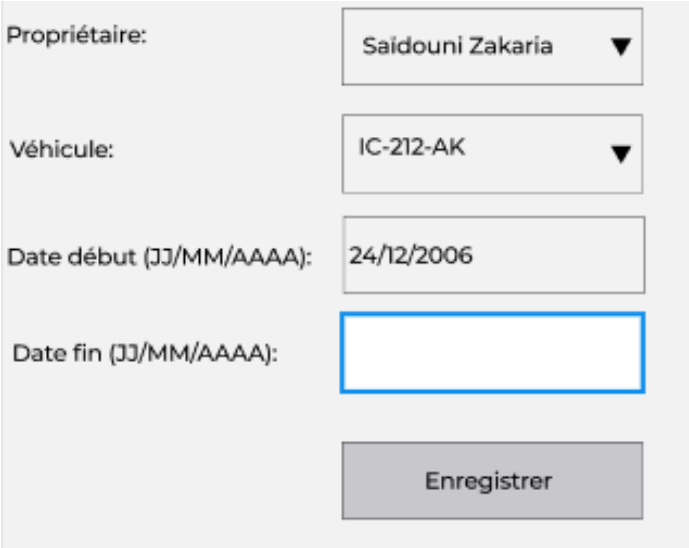


6. Conception du projet

6.1. Le front-end

6.1.1. Wireframes

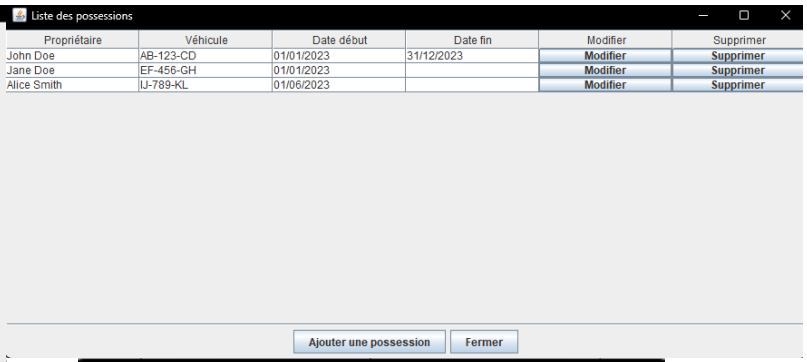


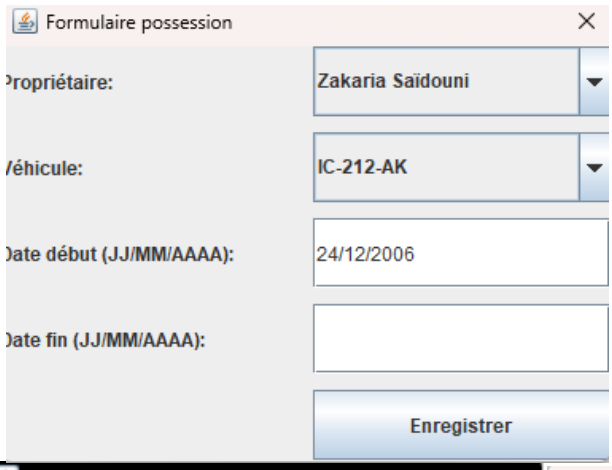


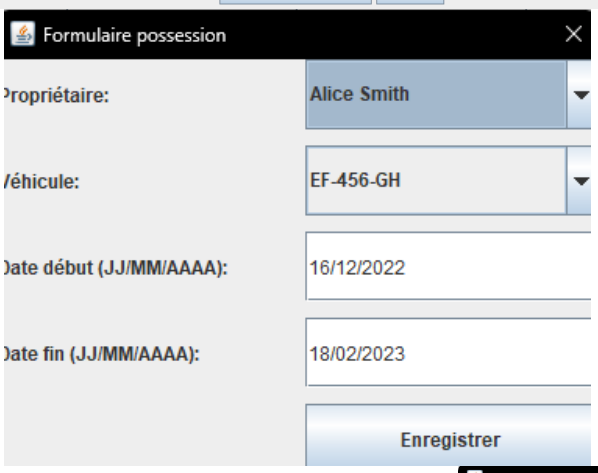
ID	Nom	Modifier	Supprimer
1	Peugeot	Modifier	Supprimer
2	Renault	Modifier	Supprimer
3	Toyota	Modifier	Supprimer

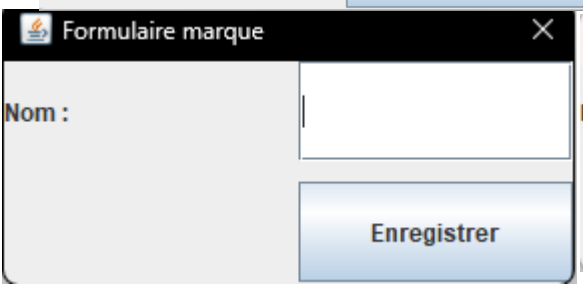
Ajouter une Marque Fermer

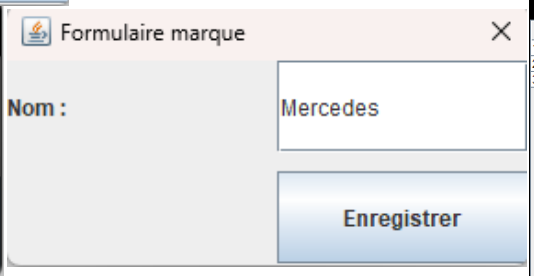
6.1.2. Maquettes



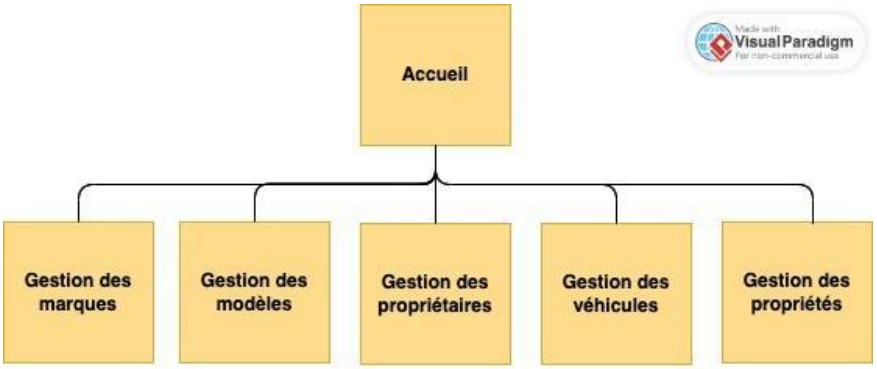






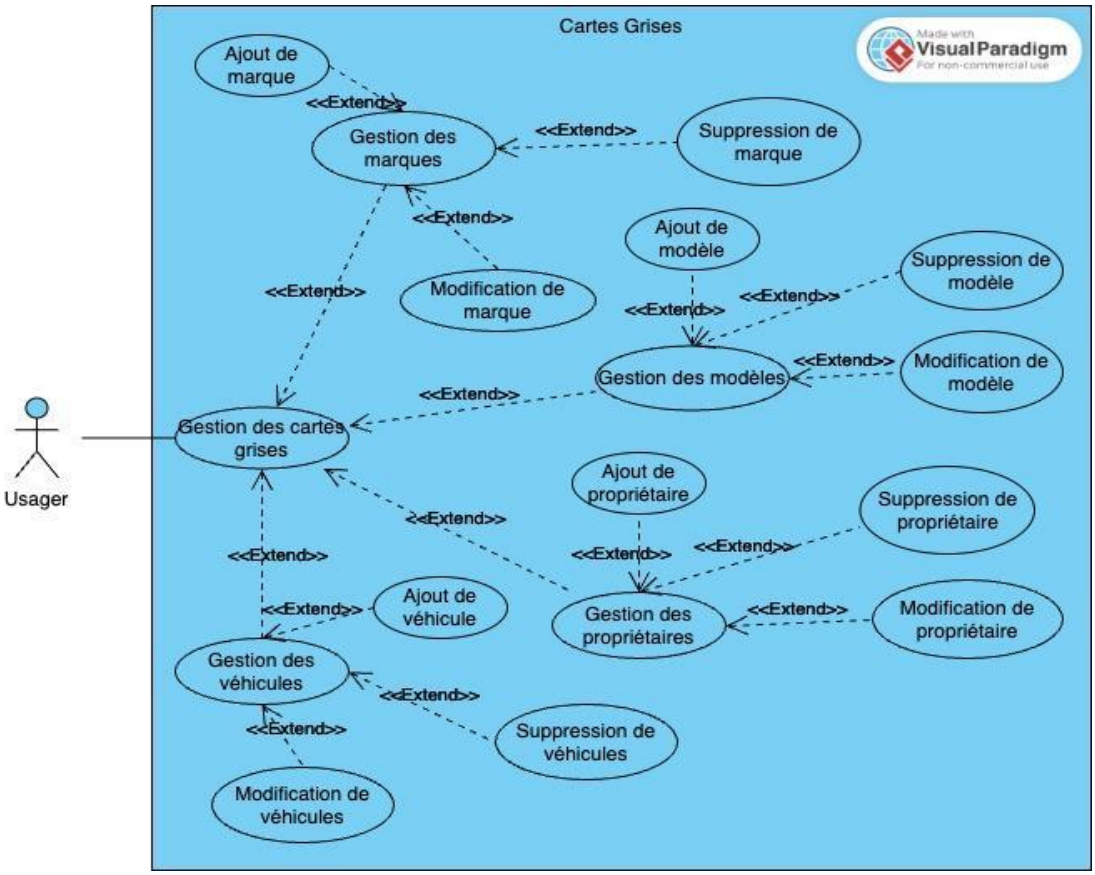


6.1.3. Arborescences

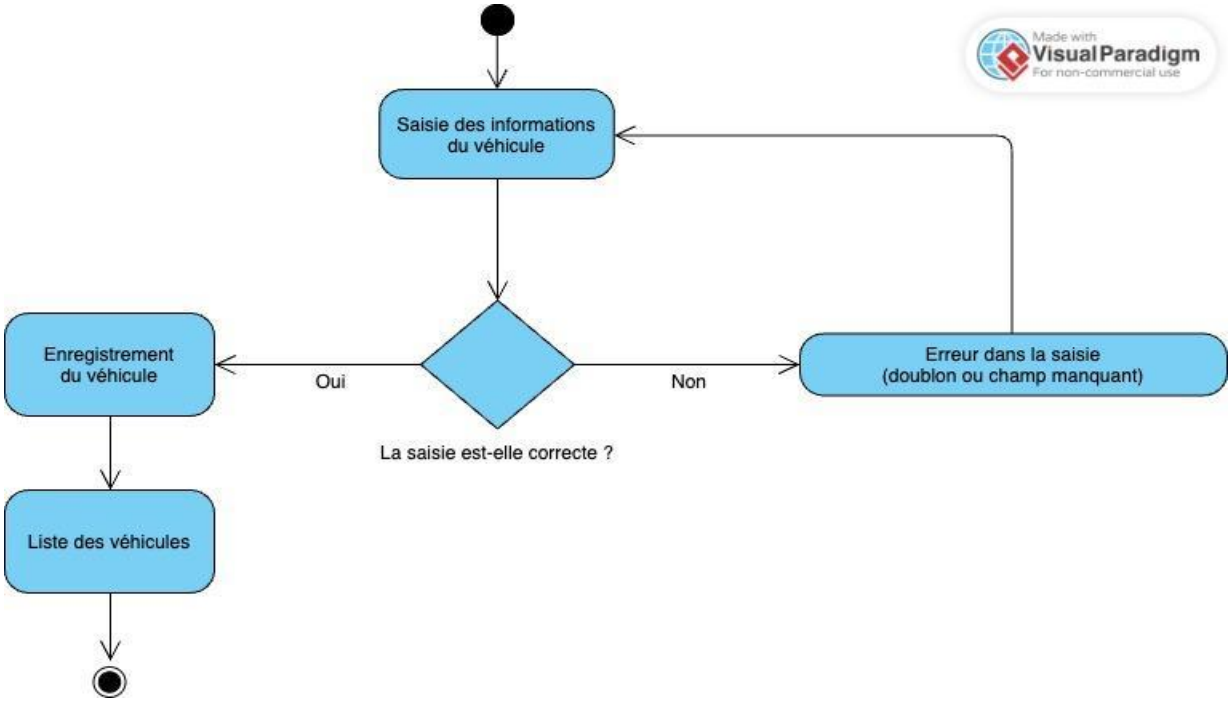


6.2. Le back-end

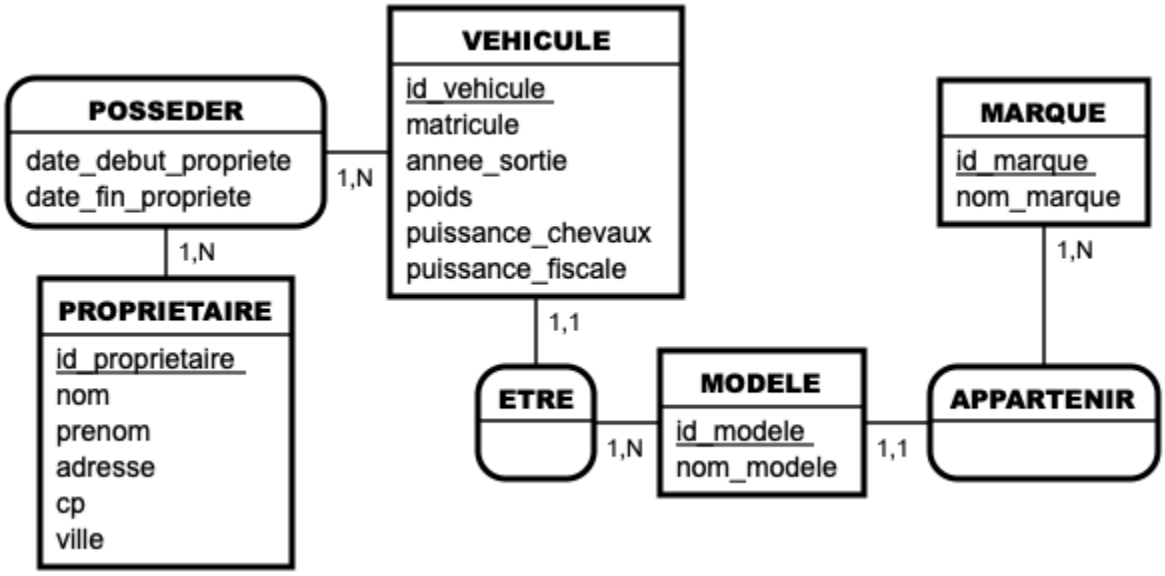
6.2.1. Diagramme de cas d'utilisation



6.2.2. Diagramme d'activités



6.2.3. Modèles Conceptuel de Données (MCD)



6.2.4. Modèle Logique de Données (MLD)

• MARQUE (id_marque, nom_marque)
Clé primaire :id_marque

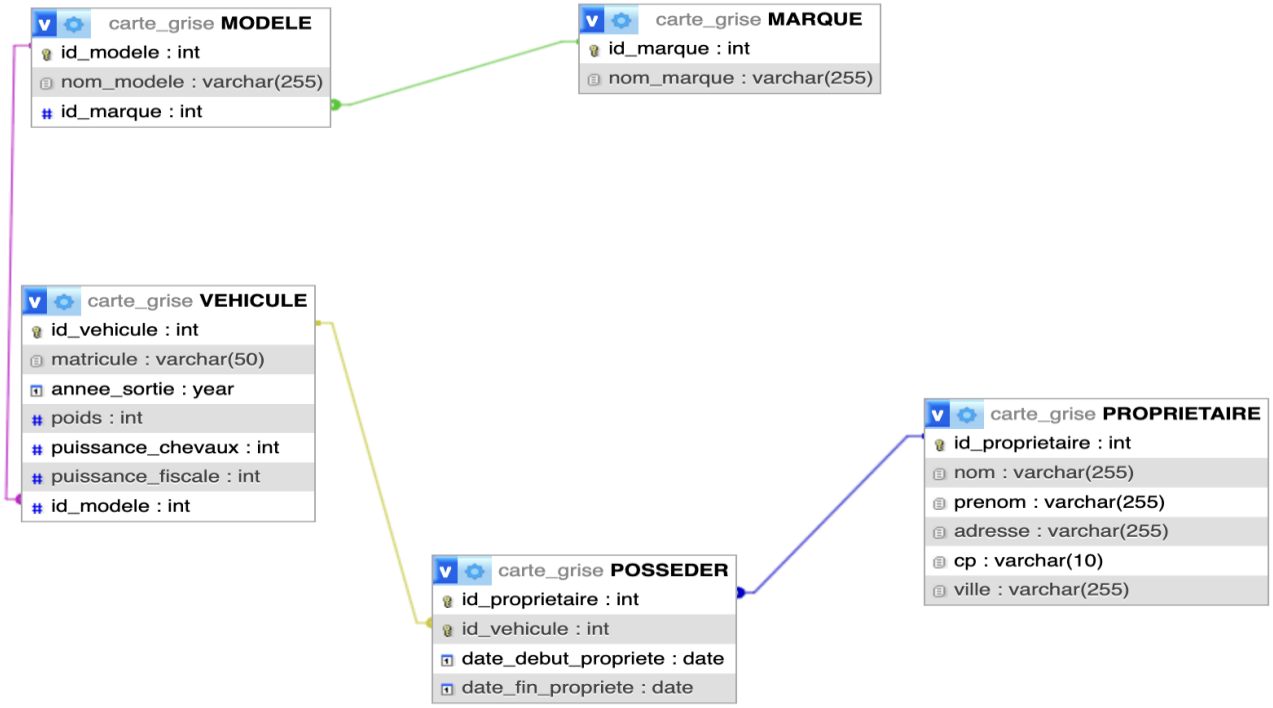
• MODELE (id_modele, nom_modele, id_marque)
Clé primaire :id_modele
Clé étrangère :id_marque en référence à id_marque de MARQUE

• VEHICULE (id_vehicule, matricule, annee_sortie, poids, puissance_chevaux, puissance_fiscale, id_modele)
Clé primaire : id_vehicule
Clé étrangère :id_modele en référence à id_modele de MODELE

• PROPRIETAIRE (id_proprietaire, nom, prenom, adresse, cp, ville)
Clé primaire :id_proprietaire

• POSSEDER (id_proprietaire, id_vehicule, date_debut_propriete, date_fin_propriete)
Clé primaire :id_proprietaire, id_vehicule
Clés étrangères :id_proprietaire en référence à id_proprietaire de PROPRIETAIRE id_vehicule en référence à id_vehicule de VEHICULE

6.2.5. Modèle Physique de Données (MPD)



7. Technologies utilisées

7.1. Langages de développement Web

7.2. Base de données

8. Sécurité

8.1. Protection contre les injections SQL

Définition de la solution :

Injection SQL : Le **SQL Injection** est une attaque qui consiste à **injecter du code SQL malveillant dans les champs de saisie** afin de manipuler la base de données.

La protection consiste à **sécuriser les requêtes SQL** pour que les données utilisateur ne soient jamais interprétées comme du code.

Extraits de code

1. REQUÊTES PRÉPARÉES DANS Vehicule.java :

```
// Ligne ~72 : Ajout d'un véhicule avec PreparedStatement
public static boolean addVehicule(String matricule, int annee, double poids,
                                  int chevaux, int fiscale, int idModele) {

    String sql = "INSERT INTO VEHICULE (matricule, annee_sortie, poids, " +
                 "puissance_chevaux, puissance_fiscale, id_modele) " +
                 "VALUES (?, ?, ?, ?, ?, ?)"; // ← Placeholders

    try (Connection conn = DBConnection.getConnection();
         PreparedStatement ps = conn.prepareStatement(sql)) {

        // Assignation sécurisée des paramètres
        ps.setString(1, matricule);
        ps.setInt(2, annee);
        ps.setDouble(3, poids);
        ps.setInt(4, chevaux);
        ps.setInt(5, fiscale);
        ps.setInt(6, idModele);

        return ps.executeUpdate() > 0;
    } catch (SQLException e) {
        System.err.println("Erreur addVehicule : " + e.getMessage());
        return false;
    }
}
```

REQUÊTE AVEC PARAMÈTRE CONDITIONNEL :

```
// Ligne ~54 : Méthode exists() avec paramètre optionnel
public static boolean exists(String matricule, Integer excludeId) {
    String sql = "SELECT COUNT(*) FROM VEHICULE WHERE matricule = ?";
    if (excludeId != null) sql += " AND id_vehicule != ?"; // ← Construction
sécurisée

    try (Connection conn = DBConnection.getConnection();
         PreparedStatement ps = conn.prepareStatement(sql)) {

        ps.setString(1, matricule);
        if (excludeId != null) ps.setInt(2, excludeId); // ← Paramètre
conditionnel

        try (ResultSet rs = ps.executeQuery()) {
            if (rs.next()) return rs.getInt(1) > 0;
        }
    } catch (SQLException e) {
        System.err.println("Erreur exists : " + e.getMessage());
    }
    return false;
}
```

TECHNIQUES UTILISÉES DANS VOTRE CODE :

- **PreparedStatement** : Séparation code/données avec ?
- **Typage fort** : setString(), setInt(), setDouble()
- **Validation métier** : Vérification existence avant action
- **Gestion transactions** : commit()/rollback() pour intégrité
- **Logs d'erreur** : Journalisation sans exposition de détails sensibles